

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	8
1.1 Ошибки мисоптимизации в компиляторах	8
1.2 Обзор существующих инструментов верификации	9
1.2.1 CompCert	9
1.2.2 Islaris	9
1.2.3 Alive2	9
1.2.4 ARM-TV	10
1.3 Сравнительный анализ и обоснование подхода	10
1.3.1 Критерии сравнения подходов	10
1.3.2 Формальная верификация и фаззинг	11
1.3.3 Обоснование выбора СНС над SMT-подходом	12
ГЛАВА 2. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ	14
2.1 Операционная семантика низкоуровневых программ	14
2.2 Архитектурная спецификация RISC-V на языке Sail	15
2.3 Ограниченные дизъюнкты Хорна	16
2.4 Кодирование семантической эквивалентности в СНС	17
2.4.1 Три нотации эквивалентности	17
2.4.2 Схема запроса	18
2.4.3 Мотивирующий пример	19
ГЛАВА 3. РЕАЛИЗАЦИЯ ИНСТРУМЕНТА RICOVER	20
3.1 Общая архитектура и конвейер	20
3.2 Трансляция Sail IR в СНС	21
3.3 Разбор RISC-V ассемблера и формирование запроса	26
3.4 Стандартная библиотека СНС	31
3.5 Модель наблюдаемого состояния	32
3.6 Кодирование ветвлений и граф потока управления	33
ГЛАВА 4. ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА	36
4.1 Тестовая среда и инструментарий	36
4.2 Тестовая база: ошибки мисоптимизации из Alive2	36
4.3 Производительность СНС-решателей	37
4.3.1 Конфигурация решателя Spacer	40
4.4 Масштабная верификация с помощью Csmith	41
4.5 Ограничения и направления развития	44
ЗАКЛЮЧЕНИЕ	46
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	48

ВВЕДЕНИЕ

Современные оптимизирующие компиляторы применяют сотни трансформаций. Корректность каждой трансформации является крайне важна: оптимизированная программа должна производить те же наблюдаемые результаты, что и исходная. Нарушение этого поведения называется мисоптимизацией и представляет особую угрозу: такая ошибка проявляется лишь при конкретных входных данных и конкретном уровне оптимизации, тогда как сам исходный код остаётся полностью корректным. Исследования показывают, что от 57 до 61 процента всех зафиксированных ошибок в компиляторах составляют именно мисоптимизации [1]. Например, инструмент формальной верификации Alive2 [2] обнаружил 47 ранее неизвестных ошибок и потребовал исправлений в официальном описании семантики LLVM IR.

Значительная часть специфичных для конкретной архитектуры процессора трансформаций осуществляется на этапе генерации кода — при переходе от промежуточного представления к конкретным инструкциям целевой архитектуры. Именно здесь инструменты верификации промежуточного уровня теряют применимость. Верифицированный компилятор CompCert [3] гарантирует корректность трансформаций внутри собственного конвейера, однако не применим к коду других компиляторов. Alive2 [2] работает исключительно на уровне LLVM IR. ARM-TV [4] проверяет переход от IR (Intermediate Representation) к AArch64, но охватывает лишь одну архитектуру и только межуровневый переход. В свою очередь, Islaris [5] верифицирует соответствие машинного кода архитектурной спецификации архитектуры, но не непосредственно трансформацию программ.

Архитектура RISC-V - это открытый стандарт набора инструкций. В отличие от проприетарных архитектур, RISC-V распространяется по открытой лицензии, что значит, что любая компания может реализовать процессор, компилятор или инструмент верификации без лицензии на такую работу. Это обусловило стремительный рост экосистемы: архитектуру поддерживают компиляторы LLVM и GCC, её внедряют в встраиваемые системы, высокопроизводительные вычисления и мобильные устройства. Крупнейшие производители полупроводников (SiFive, Western Digital, NVIDIA) выпускают процессорные ядра на базе RISC-V.

Однако для архитектуры RISC-V на сегодняшний день отсутствует инструмент автоматической формальной верификации оптимизаций

непосредственно на уровне ассемблерного кода с опорой на официальную архитектурную спецификацию.

Целью данной работы является разработка методологии и прототипа инструмента (названного RICOVER) автоматической формальной верификации семантической эквивалентности RISC-V ассемблерного кода до и после применения оптимизаций компилятора. Для достижения цели решаются следующие задачи:

- 1) Анализ подходов к переводу программ в форму ограниченных дизъюнктов Хорна;
- 2) Вычленение семантической информации инструкций из Sail модели в виде ограничений в виде дизъюнктов Хорна;
- 3) Перевод RISC-V ассемблерных инструкций в форму ограниченных дизъюнктов Хорна;
- 4) Проверка работы инструмента на подтверждённых ошибках, найденных другими инструментами;
- 5) Исследование подходов к формированию тестовой базы на основе существующих оптимизаций и её составление;
- 6) Исследование производительности решателей дизъюнктов Хорна на полученных трансляциях и интерпретация полученных результатов.

Научная новизна работы состоит в следующем: предложен метод автоматической генерации СНС-правил для инструкций RISC-V из официальной спецификации Sail [6] через Isla IR, устраняющий ручное описание семантики. Определена нотация проекционной семантической эквивалентности $(\simeq_{proj})$, разграничивающая наблюдаемое состояние (ABI-видимые регистры, внешняя память) и приватный стековый фрейм. Реализован конвейер от архитектурной спецификации и пары ассемблерных файлов до SMT-LIB2 запроса в формате ограниченных дизъюнктов Хорна (далее СНС), совместимого с решателями Z3/Spacer [7] и Eldarica [8].

Практическая значимость состоит в том, что инструмент автоматически обнаруживает мисоптимизации в RISC-V ассемблерном коде без привязки к конкретному компилятору. При обнаружении несоответствия СНС-решатель предоставляет контрпример — конкретное начальное состояние, на котором программы расходятся, что существенно упрощает отладку. Достоверность обеспечивается использованием официальной спецификации RISC-V International [6] и верификацией на примерах с заранее известным результатом.

Работа состоит из введения, четырёх глав, заключения и списка литературы. В первой главе проводится анализ предметной области и сравнение существующих инструментов верификации. Вторая глава излагает теоретические основы: операционную семантику малого шага, спецификацию Sail и формализм СНС. Третья глава описывает архитектуру и реализацию инструмента верификации. Четвёртая глава представляет результаты экспериментальной оценки на бенчмарках и Csmith.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Данная глава содержит анализ проблемы мисоптимизаций в компиляторах, обзор существующих инструментов формальной верификации оптимизаций и сравнительный анализ подходов. На основе этого анализа обосновывается выбор ограниченных дизъюнктов Хорна как целевого формализма для прототипа инструмента.

1.1 Ошибки мисоптимизации в компиляторах

Современный компилятор — это сложная многоуровневая система, выполняющая сотни трансформаций исходного кода на пути к машинным инструкциям. Каждая такая трансформация обязана сохранять семантику программы: оптимизированный вариант должен производить те же наблюдаемые результаты, что и исходный, при любых допустимых входных данных. Нарушение этого инварианта называется *мисоптимизацией* и представляет принципиально иную угрозу по сравнению с обычными ошибками компилятора: в то время как синтаксические ошибки обнаруживаются немедленно, мисоптимизация проявляется только на конкретных входных данных при конкретном уровне оптимизации, тогда как сам исходный код остаётся полностью корректным.

Статистика подтверждает серьёзность проблемы: исследования показывают, что 57-61% всех зафиксированных ошибок в компиляторах составляют именно мисоптимизации [1]. Масштаб практических последствий демонстрирует опыт инструмента Alive2 [2]: при первом применении к LLVM он обнаружил 47 ранее неизвестных ошибок оптимизаций, а его результаты потребовали внесения исправлений непосредственно в описание семантики LLVM IR.

Принципиальная трудность верификации оптимизаций состоит в том, что нужно доказать, что для *всех возможных* входных данных поведение двух программ совпадает. Ни тестирование, ни рандомизированная проверка или фаззинг не могут дать таких гарантий — они ограничены конечным множеством входов, взрывом путей исполнения и другими ограничениями. Формальная верификация, опирающаяся на математическое доказательство, является тем подходом, который способен дать исчерпывающую гарантию.

Для архитектуры RISC-V задача приобретает дополнительное измерение. Архитектура RISC-V является открытой и модульной: существует множество реализаций и компиляторов (LLVM, GCC, компиляторы для встраиваемых систем), а специфичные оптимизации кодогенератора —

сжатые инструкции, многобайтовые операции с памятью, цепочки сдвигов — осуществляются уже после того, как инструменты верификации промежуточного уровня завершили свою работу. При множестве независимых реализаций компиляторов и процессоров единственным авторитетным источником истины об архитектуре остаётся официальная формальная спецификация на языке Sail, поддерживаемая RISC-V International.

1.2 Обзор существующих инструментов верификации

1.2.1 CompCert

CompCert [3] — это верифицированный компилятор языка C, разработанный в INRIA с использованием системы доказательства теорем Coq. Ключевое свойство CompCert: все трансформации от исходного кода C до ассемблерного вывода сопровождаются машинно-проверенными доказательствами корректности.

Вместе с тем CompCert обладает принципиальным ограничением: верификация неотделима от самого компилятора. Подход не позволяет верифицировать код, произведённый другими компиляторами, и не применим к уже существующему ассемблерному коду, независимо от его происхождения.

1.2.2 Islaris

Islaris [5] — инструмент для валидации машинного кода относительно архитектурной спецификации ISA. Подход основан на символьном исполнении: инструмент извлекает трассы исполнения из машинного кода и доказывает их соответствие формальной спецификации. Islaris использует библиотеку Isla для работы с Sail IR — ту же библиотеку, которая задействована в настоящей работе.

Однако предмет Islaris — корректность реализации инструкций относительно спецификации, а не семантическая эквивалентность трансформаций программ. Кроме того, символьное исполнение порождает риск экспоненциального роста числа исследуемых путей (path explosion), что ограничивает масштабируемость подхода.

1.2.3 Alive2

Alive2 [2] — инструмент автоматической формальной верификации оптимизаций LLVM IR. Alive2 доказывает, что каждая конкретная оптимизация в компиляторе LLVM корректна: перед трансформацией и после неё он эмитирует SMT-формулы, кодирующие семантику обоих вариантов,

и задаёт вопрос о возможности расхождения. Инструмент также даёт возможность получить исходные данные, при которых оптимизация не будет сохранять эквивалентность программ.

Alive2 существенно повлиял на качество LLVM: именно благодаря ему было обнаружено 47 новых ошибок оптимизаций при первом применении. Принципиальное ограничение — Alive2 работает исключительно с LLVM IR. Это означает, что весь бэкенд компилятора — кодогенерация, выбор инструкций, планирование инструкций, оптимизации на уровне ассемблера — остаётся вне области анализа.

1.2.4 ARM-TV

ARM-TV [4] верифицирует переход от LLVM IR к коду архитектуры AArch64, тем самым закрывая часть пробела, оставленного Alive2. Инструмент переводит код AArch64 обратно в LLVM IR и применяет для сравнения алгоритмы валидации перевода.

Ограничения ARM-TV: привязанность к одной целевой архитектуре (AArch64) и анализ межуровневого перехода (IR в asm), но не внутриуровневых трансформаций внутри ассемблерного представления. Семантика AArch64 при этом не извлекается из официальной спецификации, а кодируется вручную.

1.3 Сравнительный анализ и обоснование подхода

Анализ существующих инструментов показывает: для архитектуры RISC-V отсутствует инструмент, способный автоматически верифицировать корректность трансформаций непосредственно на уровне ассемблерного кода, при этом опираясь на официальную архитектурную спецификацию как единственный источник семантики инструкций.

1.3.1 Критерии сравнения подходов

Для систематического сравнения инструментов верификации оптимизаций применяются следующие критерии:

- **Уровень анализа** — на каком представлении программы работает инструмент: исходный код, промежуточное представление (IR) или ассемблер. Для верификации оптимизаций кодогенератора требуется именно уровень ассемблера.
- **Целевая архитектура** — к каким ISA применим инструмент. Универсальный инструмент не должен быть привязан к конкретному бэкенду.

- **Источник семантики инструкций** — кодируется ли семантика вручную или извлекается из официальной архитектурной спецификации. Ручное кодирование потенциально вносит ошибки и требует актуализации при обновлении ISA.
- **Тип верифицируемых трансформаций** — межуровневые (из IR в ассемблер) или внутриуровневые. Оптимизации ассемблерного бэкенда относятся ко второму типу.
- **Поддержка циклов** — способен ли инструмент верифицировать функции с произвольными циклами без ограничения числа итераций.
- **Генерация контрпримеров** — предоставляет ли инструмент конкретное начальное состояние, на котором программы расходятся, при обнаружении ошибки. Это критически важно для отладки.
- **Уровень автоматизации** — требует ли инструмент ручной работы: задания инвариантов, аннотаций, специфичных для программы подсказок.

Сравнительный анализ по перечисленным критериям представлен в таблице 1.

Таблица 1 — Сравнение инструментов верификации оптимизаций

Инструмент	Уровень анализа	Архитектура	Семантика из спецификации	Трансформации внутри уровня
CompCert [3]	C в asm	multi	Нет (Coq-доказательства)	Да (внутри компилятора)
Alive2 [2]	LLVM IR	—	Нет	Да
ARM-TV [4]	IR в AArch64	AArch64	Нет	Нет
Islaris [5]	Asm vs. ISA	AArch64/RISC-V	Да (Sail)	Нет
RICOVER	Asm в Asm	RISC-V	Да (Sail/Isla IR)	Да

1.3.2 Формальная верификация и фаззинг

Помимо формальных методов, для обнаружения мисоптимизаций широко применяется **фаззинг** — автоматическая генерация тестовых входных данных с целью обнаружить расхождение в поведении программ. Инструменты фаззинга компиляторов (Csmith [1], YARPGen и аналоги) генерируют случайные программы и запускают их с различными уровнями оптимизации, сравнивая результаты.

Фаззинг требует перебора в **двух независимых измерениях**: необходимо генерировать и программы-примеры, и входные данные к

каждому примеру. Для обнаружения мисоптимизации нужно одновременно попасть на «правильную» программу (в которой ошибка оптимизации проявляется) и подобрать такие входные данные, на которых оптимизированный вариант расходится с оригиналом. Пространство поиска является произведением этих двух измерений, и промах по любому из них означает пропущенную ошибку. Фаззинг не даёт гарантий: даже если на 10^6 наборов входных данных расхождение не обнаружено, это не является доказательством корректности — ошибка может проявиться на $10^6 + 1$ -м наборе.

Подход на основе формальной верификации, реализованный в RICOVER, принципиально иначе разделяет пространство поиска. Генерация программ-примеров (фаззинг первого измерения) по-прежнему необходима — для этого используется Csmith. Однако **второе измерение устраняется полностью**: для каждой сгенерированной программы СНС-решатель доказывает эквивалентность на **всех возможных** входных данных, а не на конечном подмножестве. Если решатель возвращает `unsat`, это математическая гарантия того, что оптимизация корректна для данной функции при любых начальных состояниях регистров и памяти.

Таким образом, для формальной верификации нужен только перебор программ, и что самое главное - она даёт математическую гарантию того, что оптимизация корректна для всех возможных входных данных.

1.3.3 Обоснование выбора СНС над SMT-подходом

Множество инструментов, такие как, например, Alive2 используют подход, когда программа переводится в SMT-формулы, после чего решатель проверяет её выполнимость. Этот подход хорошо работает для функций без циклов, но довольно сильно ограничен при их наличии.

Над циклами в программе применяется много оптимизаций, например `loop-unroll` (размотка цикла) или `licm` (вынос инвариантного вычисления). При этом в самой программе содержится цикл с обратными рёбрами CFG. Закодировать такую программу напрямую в виде бескванторной SMT-формулы нельзя, поэтому необходимо выполнить некоторые действия:

- 1) **Раскрутка цикла** — развернуть цикл до некоего фиксированного числа k итераций. Однако, метод неполон: программа, расходящаяся на $k + 1$ итерации, останется необнаруженной. Кроме того, размер формулы растёт линейно с k .
- 2) **Квантификация** — добавить кванторы в формулу. Однако бескванторная теория (QF_BV), на которой работают эффективные SMT-

решатели, не поддерживает кванторы, а переход к полной логике первого порядка делает задачу неразрешимой или экспоненциально сложной.

- 3) **Ручное задание инварианта** — аннотировать каждый цикл инвариантом вручную. Но, это ломает автоматизацию и неприменимо к коду произвольного компилятора.

Цикл в ассемблерном коде — это базовый блок с обратным ребром CFG. В СНС-кодировании в инструменте RICOVER (раздел 3.6) такой блок генерирует **рекурсивное** правило:

$$\begin{aligned} f_from_bbN(\sigma_{in}, \sigma_{out}) &\leftarrow f_bbN(\sigma_{in}, \sigma_{mid}), \text{ cond}(\sigma_{mid}), \\ f_from_bbN(\sigma_{mid}, \sigma_{out}) \end{aligned}$$

Это правило говорит нам: «начав итерацию в состоянии σ_{in} , выполнив тело блока, при выполнении условия продолжаем с нового состояния σ_{mid} ». СНС-решатель (в нашем случае Spacer [7]) ищет некий **индуктивный инвариант** — предикат $Inv(\sigma)$, замкнутый относительно этого правила. Найденный инвариант является математическим доказательством корректности для **всех** итераций цикла, а не только для первых k .

Таким образом, СНС является не просто альтернативным форматом, а качественно другим уровнем выразительности: рекурсивные правила позволяют кодировать семантику программ с циклами, делегируя поиск инварианта специализированному алгоритму решателя.

Кроме того, выбор ограниченных дизъюнктов Хорна (СНС) как целевого формализма обоснован тремя соображениями. Во-первых, СНС являются стандартным промежуточным форматом для верификации императивных программ [9]: они выражают как операционную семантику (правила переходов), так и свойства корректности (ограничения-запросы) в единой системе. Во-вторых, современные СНС-решатели (Z3/Spacer [7], Eldarica [8]) поддерживают теории массивов и битовых векторов, необходимые для точного кодирования регистрового файла и памяти. В-третьих, СНС-формат HORN является стандартом SMT-LIB2, что обеспечивает совместимость с различными решателями без изменения кода инструмента.

ГЛАВА 2. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

Данная глава излагает формальные основания инструмента RICOVER: операционную семантику малого шага для низкоуровневых программ, архитектурную спецификацию RISC-V на языке Sail, формализм ограниченных дизъюнктов Хорна и кодирование семантической эквивалентности в CHC.

2.1 Операционная семантика низкоуровневых программ

Для определения понятия семантической эквивалентности двух программ необходимо прежде всего формализовать понятие *семантики программы* — что именно программа «делает». В данной работе используется *операционная семантика малого шага* (small-step operational semantics), также известная как структурная операционная семантика [10].

В операционной семантике малого шага поведение программы описывается как последовательность элементарных переходов между *конфигурациями*. Конфигурация — это пара $\langle s, \sigma \rangle$, где s — остаток программы к исполнению (текущая инструкция), σ — текущее состояние машины.

Определение 2.1 (Шаг исполнения). Отношение *одного шага* $\Rightarrow$ задаётся правилами перехода:

$$\langle s_0, \sigma_0 \rangle \Rightarrow \langle s_1, \sigma_1 \rangle \quad (2.1)$$

где s_0 — текущая инструкция, исполняемая в состоянии σ_0 ; s_1 — следующая инструкция, σ_1 — новое состояние после применения семантики s_0 .

Определение 2.2 (Полное исполнение — exes). Полное исполнение программы — достижение завершающего состояния halt из начального состояния σ_0 :

$$\text{exes}(P, \sigma_0, \sigma_1) \leftarrow \langle P, \sigma_0 \rangle \stackrel{*}{\Rightarrow} \langle \text{halt}, \sigma_1 \rangle \quad (2.2)$$

где $\stackrel{*}{\Rightarrow}$ — рефлексивное транзитивное замыкание отношения $\Rightarrow$; псевдооператор halt обозначает нормальное завершение программы (при его достижении счётчик команд не обновляется, что является стандартным приёмом в малошаговой семантике).

В CHC-нотации цепочка шагов и полное исполнение кодируются тремя правилами:

$$\text{run}(S, \sigma, S, \sigma) \leftarrow \top \quad (2.3)$$

$$\text{run}(S_0, \sigma_0, S_2, \sigma_2) \leftarrow \text{step}(S_0, \sigma_0, S_1, \sigma_1), \text{run}(S_1, \sigma_1, S_2, \sigma_2) \quad (2.4)$$

$$\text{exec}(S, \sigma_0, \sigma_1) \leftarrow \text{run}(S, \sigma_0, \text{halt}, \sigma_1) \quad (2.5)$$

Применительно к RISC-V ассемблеру состояние машины σ — это тройка:

$$\sigma = \langle \text{pc}, \text{regs}, \text{mem} \rangle \quad (2.6)$$

где $\text{pc} \in \mathbb{B}^{64}$ — 64-битный счётчик команд; $\text{regs} : \mathbb{B}^5 \rightarrow \mathbb{B}^{64}$ — функция, отображающая пятибитный индекс регистра в 64-битное значение; $\text{mem} : \mathbb{B}^{64} \rightarrow \mathbb{B}^8$ — побайтовое отображение адресного пространства памяти. Именно эта модель состояния используется в СНС-стандартной библиотеке RICOVER (`stdlib.smt2`).

Вместо того чтобы задавать функцию `step` вручную для каждой инструкции, в настоящей работе она извлекается из официальной архитектурной спецификации RISC-V — как описано в следующем разделе.

2.2 Архитектурная спецификация RISC-V на языке Sail

Sail — это специализированный язык для описания семантики архитектур набора инструкций (ISA) [11]. Формальная спецификация RISC-V на языке Sail, разработанная и принятая организацией RISC-V International [6], является официальным машиночитаемым описанием архитектуры.

Спецификация определяет функцию `execute` для каждой инструкции. Например, для класса целочисленных инструкций с непосредственным операндом (ITYPE) спецификация содержит определение, показанное в листинге 2.1.

```

1  function clause execute ITYPE(imm, rs1, rd, op) = {
2      let immext : xlenbits = sign_extend(imm);
3      X(rd) = match op {
4          ADDI    => X(rs1) + immext,
5          SLTI    => zero_extend(bool_to_bit(X(rs1) <_s immext)),
6          ANDI    => X(rs1) & immext,
7          ORI     => X(rs1) | immext,
8          XORI    => X(rs1) ^ immext
9      };
10     RETIRE_SUCCESS
11 }
```

Листинг 2.1 — Функция `execute ITYPE` в спецификации Sail

Это описание задаёт семантику инструкций ADDI, SLTI, ANDI, ORI, XORI. А именно: какой регистр модифицируется, каким образом, что происходит с памятью и счётчиком команд.

Для работы с Sail в RICOVER используется библиотека **Isla** (isla-lib) [11]. Isla — это символьная машина, способная исполнять Sail-код и транслировать его в формальное представление. Инструмент Isla-sail компилирует Sail код описания архитектуры RISC-V в *Isla IR* — промежуточное представление, из которого гораздо удобнее вычленять необходимую нам информацию. Именно этот файл .ir служит входными данными для модуля isla_ir.rs в RICOVER.

Ключевой шаг после получения Isla IR — *специализация* вариантов. Функция execute — это один обработчик на несколько вариантов через сравнение с образцом. Isla IR сохраняет эту структуру, а isla_ir.rs перебирает все ветки и специализирует каждую по конкретному значению *op*, получая отдельное СНС-правило на каждый вариант инструкции. Таким образом, одна Sail-функция execute ITYPE порождает шесть независимых СНС-правил: addi, slti, sltiu, andi, ori, xori.

Полный вывод в виде Isla IR для RISC-V 64 (файл snapshot/rv64d.ir) содержит 746 вариантов инструкций. Из них 210 успешно транслируются в СНС-правила, поддерживая расширенный профиль rv64i.

2.3 Ограниченные дизъюнкты Хорна

Ограниченные дизъюнкты Хорна (Constrained Horn Clauses, СНС) — это фрагмент логики первого порядка, в котором дизъюнкты Хорна расширяются использованием формул произвольной теории ограничений [9]. СНС обеспечивают пригодную промежуточную форму для выражения семантики различных языков программирования и вычислительных моделей [12].

Определение 2.3 (Ограниченный дизъюнкт Хорна). СНС — это формула вида:

$$\varphi \wedge p_1(\vec{V}) \wedge \dots \wedge p_k(\vec{V}) \Rightarrow H \quad (2.7)$$

где φ — *ограничение* в теории A (битовые векторы, массивы, целые числа); $p_1, \dots, p_k$ — *неинтерпретированные предикатные символы*; $\vec{V}$ — набор переменных; H — либо применение предиката $p_i(\vec{V})$, либо false.

Если $H = \text{false}$, дизъюнкт называется *запросом* (integrity constraint). Если тело пусто ($k = 0$, φ — тавтология), дизъюнкт называется *фактом*.

Теория ограничений A в RICOVER — это теория битовых векторов (QF_BV) в сочетании с теорией массивов (QF_ABV), что позволяет точно моделировать 64-битные регистры и побайтовую адресуемую память.

Дизъюнкты Хорна естественно кодируют множество достижимых состояний последовательных программ: *выполнимые* дизъюнкты Хорна соответствуют свойствам программы, которые выполняются, тогда как *невыполнимые* — нарушенным свойствам. Это делает СНС канонической промежуточной формой для верификации программ.

Схема верификации свойств безопасности через СНС выглядит следующим образом. Для системы с начальным состоянием $\text{init}(v)$ и шагом $\text{step}(v, v')$ ищется *индуктивный инвариант* $\text{inv}(v)$:

$$\text{inv}(v) \leftarrow \text{init}(v) \quad (2.8)$$

$$\text{inv}(v') \leftarrow \text{inv}(v) \wedge \text{step}(v, v') \quad (2.9)$$

$$\text{safe}(v) \leftarrow \text{inv}(v) \quad (2.10)$$

СНС-решатель проверяет выполнимость системы: если safe выводимо — свойство доказано; если не выводимо — найден контрпример.

Для решения СНС в настоящей работе применяется решатель Z3 Spacer [7].

2.4 Кодирование семантической эквивалентности в СНС

Различные инструменты представляют семантическую эквивалентность в разных форматах. Для нас релевантно оценить три следующих варианта: полная, ABI и проекционная эквивалентность.

2.4.1 Три нотации эквивалентности

Для двух RISC-V функций P и Q можно определить три нотации семантической эквивалентности в порядке убывания строгости.

Нотация 1: Полная эквивалентность состояний ($\simeq_{\text{full}}$).

Две программы полностью эквивалентны, если для любого начального состояния σ_0 их конечные состояния совпадают полностью:

$$P \simeq_{\text{full}} Q \Leftrightarrow \forall \sigma_0, \sigma_1, \sigma_2. (\text{exes}(P, \sigma_0, \sigma_1) \wedge \text{exes}(Q, \sigma_0, \sigma_2)) \rightarrow \sigma_1 = \sigma_2 \quad (2.11)$$

где σ_0 — начальное состояние, σ_1, σ_2 — конечные состояния функций P и Q соответственно.

СНС-запрос для данной нотации будет выглядеть так:

$$\text{bad}_{\text{full}} \leftarrow \text{exec}_P(\sigma_0, \sigma_1), \text{exec}_Q(\sigma_0, \sigma_2), \sigma_1 \neq \sigma_2 \quad (2.12)$$

Однако, эмперическим путём, было обнаружено, что эта нотация слишком строга для верификации оптимизаций: оптимизации почти всегда изменяют стек: размещение временных переменных, порядок сохранения регистров и т.п. Поэтому bad_{full} оказывается выполнимым даже для семантически корректных оптимизаций.

Нотация 2: ABI-эквивалентность ($\simeq_{\text{abi}}$).

Более практичная нотация: две программы ABI-эквивалентны, если они согласуются по всем видимым для вызывающего кода характеристикам:

$$P \simeq_{\text{abi}} Q \Leftrightarrow \forall \sigma_0, \sigma_1, \sigma_2. (\text{exec}(P, \sigma_0, \sigma_1) \wedge \text{exec}(Q, \sigma_0, \sigma_2)) \rightarrow \text{Obs}_{\text{abi}}(\sigma_1) = \text{Obs}_{\text{abi}}(\sigma_2) \quad (2.13)$$

где функция Obs_{abi} возвращает проекцию на ABI-видимые регистры: возвращаемое значение (регистр a0), сохраняемые регистры (ra, sp, s0–s11) и программный счётчик. Этот набор определяется соглашением о вызовах RISC-V (RISC-V ABI) [6].

Нотация 3: Проекционная эквивалентность ($\simeq_{\text{proj}}$).

Наиболее сбалансированная нотация расширяет ABI-эквивалентность требованием о согласованности *наблюдаемой памяти*:

$$P \simeq_{\text{proj}} Q \Leftrightarrow \forall \sigma_0, \sigma_1, \sigma_2. (\text{exec}(P, \sigma_0, \sigma_1) \wedge \text{exec}(Q, \sigma_0, \sigma_2)) \rightarrow (\text{Obs}_{\text{abi}}(\sigma_1) = \text{Obs}_{\text{abi}}(\sigma_2) \wedge \forall a \in \text{ObsMem}(\sigma_0). \text{mem}_1[a] = \text{mem}_2[a]) \quad (2.14)$$

где $\text{ObsMem}(\sigma_0)$ — множество *наблюдаемых* адресов памяти, то есть все адреса за исключением приватного стекового фрейма функции. Фрейм вычисляется как интервал $[\text{sp}_0 - N, \text{sp}_0]$, где N извлекается из первой инструкции функции вида `addi sp, sp, -N`; mem_1 и mem_2 — конечные состояния памяти функций P и Q соответственно.

2.4.2 Схема запроса

Конкретный СНС-запрос для $\simeq_{\text{proj}}$ формулируется через отношение bad , выводимое ровно тогда, когда существует начальное состояние, при котором программы расходятся:

$$\text{bad} \leftarrow P(\sigma_0, \sigma_1), Q(\sigma_0, \sigma_2), \text{Obs}_{\text{abi}}(\sigma_1) \neq \text{Obs}_{\text{abi}}(\sigma_2) \quad (2.15)$$

$$\text{bad} \leftarrow P(\sigma_0, \sigma_1), Q(\sigma_0, \sigma_2), a \in \text{ObsMem}(\sigma_0), \text{mem}_1[a] \neq \text{mem}_2[a] \quad (2.16)$$

Далее СНС-решатель проверяет выводимость bad :

- **unsat** — bad недостижимо, обе программы семантически эквивалентны относительно $\underset{\text{proj}}{\simeq}$;
- **sat** — существует начальное состояние, при котором программы расходятся; решатель предоставляет конкретный контрпример.

Именно нотация $\underset{\text{proj}}{\simeq}$ реализована в инструменте RICOVER и применяется во всех экспериментах данной работы.

2.4.3 Мотивирующий пример

Рассмотрим функцию, реализующую вычисление $y = x + 2$ при $x = 1$ (листинги 2.2 и 2.3).

```

1  addi    sp, sp, -32
2  sd      ra, 24(sp)
3  sd      s0, 16(sp)
4  addi    s0, sp, 32
5  addi    a0, zero, 1
6  sw      a0, -20(s0)
7  lw      a0, -20(s0)
8  addiw   a0, a0, 2
9  sw      a0, -24(s0)
10 lw      a0, -24(s0)
11 ld      ra, 24(sp)
12 ld      s0, 16(sp)
13 addi    sp, sp, 32
14 ret

```

Листинг 2.2 — Функция foo1 — неоптимизированный вариант (RISC-V ассемблер)

```

1  addi    a0, zero, 3
2  ret

```

Листинг 2.3 — Функция foo2 — оптимизированный вариант (RISC-V ассемблер)

Запрос bad_{full} для этих двух функций **выполним** (sat): финальная память отличается, поскольку foo1 записала временные значения на стек. Запрос по формулам выше **невыполним** (unsat): записи выполнены только в приватный фрейм $[\text{sp}_0 - 32, \text{sp}_0)$, который исключён из ObsMem, а регистр a0 в обоих случаях равен 3. Это и есть правильный ответ: константная свёртка семантически корректна.

ГЛАВА 3. РЕАЛИЗАЦИЯ ИНСТРУМЕНТА RICOVER

3.1 Общая архитектура и конвейер

RICOVER (RISC-V Compiler Optimization VERification) — инструмент формальной верификации, реализованный на языке Rust. Он принимает два RISC-V ассемблерных файла и снимок Isla IR RISC-V модели, и транслирует это в формат SMT-LIB2, формируя CHC-запрос. Архитектура конвейера представлена на рисунке 3.1.

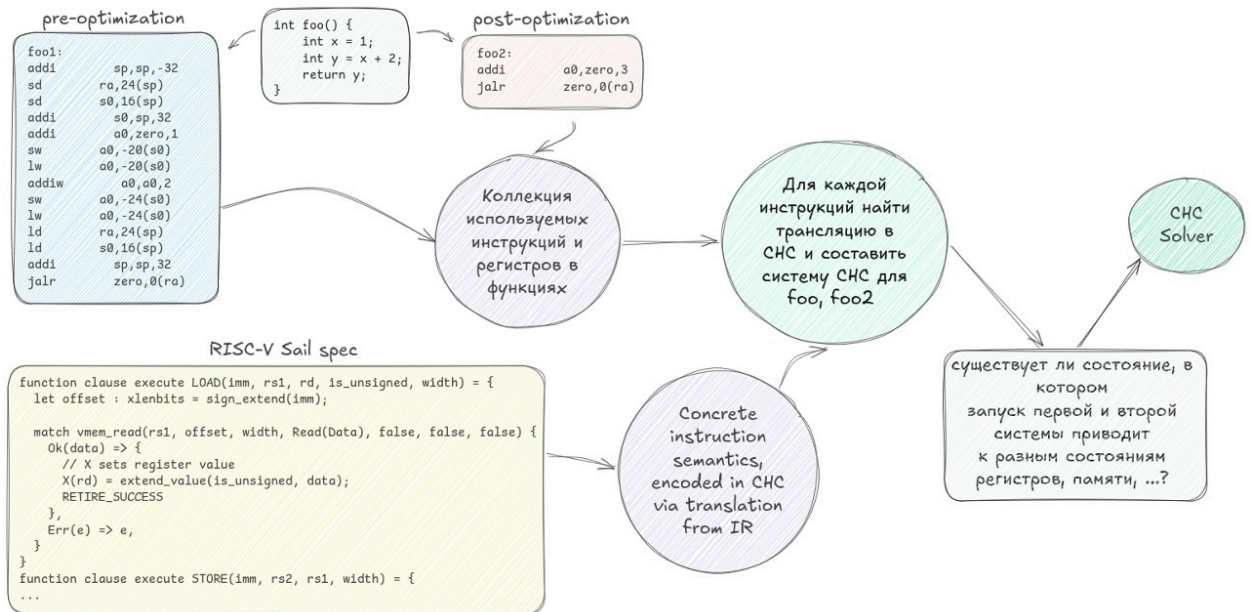


Рисунок 3.1 — Архитектура конвейера RICOVER

Проект организован следующим образом:

- `src/main.rs` — точка входа CLI (Clap), маршрутизирует команды `translate-ir` и `check-equiv`;
- `src/isla_ir.rs` — загружает `.ir` файл через `isla-lib`, извлекает тела `execute`-блозов, специализирует варианты и транслирует в CHC;
- `src/asm_parse.rs` — разбирает RISC-V ассемблерные файлы в структурированные последовательности инструкций;
- `src/chc_emit/` — формирует финальный CHC-запрос: соединяет CHC-правила инструкций в цепочку состояний и добавляет запрос на эквивалентность;
- `chc_stdlib/stdlib.smt2` — CHC стандартная библиотека.

Точка входа RICOVER определяет две подкоманды через интерфейс командной строки. Листинг 3.1 демонстрирует их объявление.

```
1 /// RICOVER - RISC-V Compiler Optimization
  Verification via CHC
```

Rust

```

2  #[derive(Parser)]
3  #[command(name = "ricover")]
4  enum Cli {
5      /// Translate Isla IR instruction definitions to CHC
6      TranslateIR {
7          #[arg(short, long)] ir_file: PathBuf,
8          #[arg(short, long)] output: PathBuf,
9          #[arg(short, long)] functions: Vec<String>,
10     },
11
12     /// Emit CHC equivalence query from two RISC-V
13     assembly functions
14     CheckEquiv {
15         #[arg(long)] before: PathBuf,
16         #[arg(long)] after: PathBuf,
17         #[arg(short, long)] function: String,
18         #[arg(long)] before_fn: Option<String>,
19         #[arg(long)] after_fn: Option<String>,
20         #[arg(short, long)] output: PathBuf,
21         #[arg(long)] ir: PathBuf,
22     },
23 }

```

Листинг 3.1 — Перечисление команд CLI в main.rs

RICOVER поддерживает две команды:

- 1) translate-ir — транслирует execute-функции из .ir файла в CHC-правила инструкций;
- 2) check-equiv — строит полный запрос на эквивалентность двух функций.

3.2 Трансляция Sail IR в CHC

Основная задача модуля isla_ir.rs — преобразовать тела execute-функций Isla IR в CHC-правила, формально кодирующие семантику каждой инструкции RISC-V.

Загрузка IR выполняется через публичные структуры данных модуля. Структура IslaIRModel хранит разобранные определения IR и таблицу символов; IrFunction представляет одну функцию с её телом (листинг 3.2).

```

1  pub struct IslaIRModel<'ir> {
2      pub defs: Vec<Def<Name, B129>>,

```

 Rust

```

3      pub symtab: Symtab<'ir>,
4  }
5
6  pub struct IrFunction<'a> {
7      pub name: String,
8      pub params: Vec<Name>,
9      pub body: &'a [Instr<Name, B129>],
10 }

```

Листинг 3.2 — Структуры данных модуля isla_ir.rs (Rust)

Загрузка и разбор .ir файла выполняются функцией `parse_ir`, которая делегирует синтаксический анализ парсеру isla-lib (листинг 3.3).

```

1  pub fn parse_ir(contents: &str) ->
    Result<IslaIRModel<'_>> {
2      let mut symtab = Symtab::new();
3      let defs = IrParser::new()
4          .parse(&mut symtab, new_ir_lexer(contents))
5          .map_err(|e| anyhow!("IR parse error: {}", e))?;
6      Ok(IslaIRModel { defs, symtab })
7  }

```

Листинг 3.3 — Загрузка и разбор Isla IR через isla-lib (Rust)

Алгоритм трансляции состоит из четырёх шагов.

- 1) **Загрузка IR.** Функция `read_ir_file` читает бинарный .ir файл с диска; `parse_ir` десериализует содержимое в AST Isla IR (листинг 3.3). Модуль ищет функции с именем `execute` (или указанные через флаг `-f`).
- 2) **Обнаружение вариантов** (`discover_variants`). Тело функции `execute` содержит `match`-выражения по типу инструкции. Модуль рекурсивно обходит IR, сопоставляя ветки `match` с конкретными вариантами инструкций (`ADDI`, `LOAD`, `BRANCH` и т. д.).
- 3) **Специализация варианта** (`emit_variant_chc`). Для каждого варианта: параметры инструкции (поля операнда, регистровые номера) становятся свободными переменными СНС-правила, каждое присваивание регистра кодируется через вызов `set_reg(regs0, rd, val)`, а обращения к памяти — через `mem_read_N/write_mem_*`. Также обновляем PC, если инструкция не осуществляет переход, то `pc1 = bvadd(pc0, 4)`.
- 4) **Оптимизация: устранение переменных памяти.** Инструкции, не изменяющие память, например, арифметические, логические - не

получают собственной переменной памяти memN. Вместо этого используется предыдущая переменная memK, устраняя некоторые тривиальные ограничения вида (= memN memN-1) над массивами.

Результатом шага 3 является СНС-правило для каждой инструкции. Листинг 3.4 демонстрирует правило для `addi rd, rs1, imm`, полученное из Sail функции `execute ITYPE` через Isla IR.

```

1  (declare-rel addi
2    ((Array (_ BitVec 5) (_ BitVec 64))
3     (Array (_ BitVec 64) (_ BitVec 8))
4     (_ BitVec 64)
5     (Array (_ BitVec 5) (_ BitVec 64))
6     (Array (_ BitVec 64) (_ BitVec 8))
7     (_ BitVec 64)
8     (_ BitVec 12) (_ BitVec 5) (_ BitVec 5)))
9
10 (rule
11   (forall ((regs0 (Array (_ BitVec 5) (_ BitVec 64)))
12            (mem0   (Array (_ BitVec 64) (_ BitVec 8)))
13            (pc0    (_ BitVec 64))
14            (regs1  (Array (_ BitVec 5) (_ BitVec 64)))
15            (mem1   (Array (_ BitVec 64) (_ BitVec 8)))
16            (pc1    (_ BitVec 64))
17            (p0 (_ BitVec 12)) (p1 (_ BitVec 5)) (p2 (_ BitVec 5)))
18     (=> (and
19          (= regs1 (set_reg regs0 p2
20                        (bvadd (get_reg regs0 p1)
21                              ((_ sign_extend 52)
22                               p0))))
22          (= mem1 mem0)
23          (= pc1 (bvadd pc0 (_ bv4 64))))
24          (addi regs0 mem0 pc0 regs1 mem1 pc1 p0 p1
25                p2))))

```

Листинг 3.4 — Транслированное СНС-правило инструкции `addi` (в формате SMT-LIB2)

Читать это правило следует так: если мы начали из состояния (`regs0`, `mem0`, `pc0`), то исполнение `addi` с операндами `imm=p0`, `rs1=p1`, `rd=p2`

приводит к состоянию (`regs1`, `mem1`, `pc1`), где регистр `rd` получает значение `rs1 + sign_extend(imm)`. Память не изменяется, а PC увеличивается на 4. Тело правила было сгенерировано инструментом из Sail функции через Isla IR — оно не написано вручную.

Команда `translate-ir` генерирует для каждой инструкции отдельное отношение (`declare-rel + rule`), как показано в листинге 3.4. Вместо обращения к каждому сгенерированному отношению, шаг `check-equiv` применяет прямое встраивание, то есть, тело каждого правила блока содержит три равенства-ограничения непосредственно, без ссылки на инструкцию в виде отношения. Такой подход сокращает глубину системы с двух уровней (инструкционные отношения, программные отношения) до одного (только блочные и композиционные отношения), что упрощает задачу поиска индуктивного инварианта алгоритмом `Spacer`.

Технически инлайнинг реализован через структуру `InlinedSemantics` (листинг 3.5) — три строковых шаблона, хранящих SMT-выражения для нового регистрового файла, памяти и PC.

```
1  pub(crate) struct InlinedSemantics { Rust
2      /// Шаблон для regs_out. Содержит "regs0" и
3      /// "p0".."pN".
4      pub regs_expr: String,
5      /// Шаблон для mem_out. "mem0" если память не
6      /// меняется.
7      pub mem_expr: String,
8      /// Шаблон для pc_out. Обычно "(bvadd pc0 (_ bv4
9      64))".
10     pub pc_expr: String,
11     pub n_params: usize,
12 }
13
14 impl InlinedSemantics {
15     pub fn instantiate_compressed(
16         &self, operands: &[String], si: usize, so: usize,
17         mi: usize, mo: usize,
18     ) -> Vec<String> {
19         let mut regs_e = self.regs_expr.replace("regs0",
20             &format!("regs{si}"));
21         let mut mem_e = self.mem_expr.replace("mem0",
22             &format!("mem{mi}"));
```

```

18         let mut pc_e      = self.pc_expr      .replace("pc0",
19         &format!("pc{si}"));
20         for i in (0..self.n_params).rev() {
21             for expr in [&mut regs_e, &mut mem_e, &mut
22             pc_e] {
23                 *expr = expr.replace(&format!("p{i}"),
24                 &operands[i]);
25             }
26         }
27         let mut out = vec![
28             format!("(= regs{so} {regs_e})"),
29             format!("(= pc{so} {pc_e})"),
30         ];
31         if mi != mo { out.insert(1, format!("(= mem{mo}
32         {mem_e})")); }
33         out
34     }
35 }

```

Листинг 3.5 — Структура InlinedSemantics и метод `instantiate_compressed` (`ir_emit.rs`, Rust)

При построении правила блока `emit_block_body_rule_inlined` вызывает `instantiate_compressed` для каждой инструкции: результирующие три строки добавляются непосредственно в тело (`and ...`). Когда `mi == mo` (инструкция не изменяет память), ограничение `(= memN memN)` опускается — это и есть оптимизация устранения переменных памяти, совмещённая с инлайнингом в одном проходе.

Из 746 вариантов в полном `Isla IR rv64d.ir` успешно транслируются 199. Покрытые инструкции включают полный профиль RV64IM: ITYPE (`addi/addiw/andi/ori/xori/slti/sltiu`), RTYPE (`add/sub/and/or/xor` + W-варианты, сдвиги), LOAD (`lb/lbu/lh/lhu/lw/lwu/ld`), STORE (`sb/sh/sw/sd`), BRANCH (`beq/bne/blt/bge/bltu/bgeu`), JAL, JALR, LUI, MUL/MULH/DIV/REM и их варианты. Остальные 547 относятся к расширениям с плавающей точкой (F, D, H), векторным (V), криптографическим (K) и привилегированным (M/S) инструкциям — для целочисленных программ, генерируемых Csmith, все используемые инструкции покрыты.

Формирование SMT-LIB2-вывода сосредоточено в модуле `src/chc_emit/`, организованном в виде восьми специализированных подмодулей (листинг 3.6).

```

1  mod format;                // текстовое
   представление SMT-выражений
2  mod smt;                   // трансляция Isla IR Exp ->
   SMT-строка
3  mod known_calls;           // классификация известных Sail
   вызовов
4  mod variant_discovery;     // обнаружение вариантов execute в
   IR-дереве
5  mod ir_translate;          // PathTranslation: обход пути
   варианта
6  mod ir_emit;               // InlinedSemantics: шаблоны
   ограничений
7  mod asm_emit;              // emit_program_rule_inlined:
   правила блоков
8  mod equiv;                 // emit_equivalence_query: сборка
   запроса
9
10 pub use ir_emit::emit_instruction_chc;
11 pub use equiv::emit_equivalence_query;

```

Листинг 3.6 — Объявление подмодулей `chc_emit` (`src/chc_emit/mod.rs`, Rust)

Ключевой поток данных через модули: `variant_discovery` находит все пути выполнения в IR для каждого варианта инструкции в `ir_translate` обходит каждый путь и строит структуру `PathTranslation` в `ir_emit` преобразует `PathTranslation` в `InlinedSemantics` — готовые SMT-шаблоны в `asm_emit` применяет шаблоны к конкретным ассемблерным операндам при построении правил блоков. Функция `emit_equivalence_query` в `equiv.rs` является точкой сборки.

3.3 Разбор RISC-V ассемблера и формирование запроса

Команда `check-equiv` принимает два ассемблерных файла, имя функции, путь к `.ir` и выходной файл.

Разборщик читает GAS-совместимые `.s` файлы и строит для каждой функции список структурированных инструкций. Поддерживаются базовые инструкции RV64GC, псевдоинструкции (`ret` в `jalr x0, 0(ra)`, `mv` в `addi rd, rs, 0`, `li`, `snez`, `zext.b`, `lui`) и сжатые инструкции (`c.addi`, `c.li`, `c.mv`, `c.ld`, `c.sd`, `c.beqz`, `c.bnez`), раскрываемые в базовые при разборе.

Разбор основан на пяти структурах данных, отображающих элементы ассемблерного текста на типизированные объекты Rust (листинг 3.7).

```

1  pub struct AsmInstruction {
2      pub opcode: String,
3      pub operands: Vec<Operand>,
4  }
5
6  pub enum Operand {
7      Reg(String), //
        регистр: "sp", "a0", "ra"
8      Imm(i64), //
        непосредственный операнд: -32
9      MemRef { offset: i64, base: String }, // ссылка:
        -20(s0), 24(sp)
10     Label(String), // цель
        ветвления: ".LBB0_2"
11 }
12
13 pub struct AsmFunction {
14     pub name: String,
15     pub instructions: Vec<AsmInstruction>,
16     pub labels: HashMap<String, usize>,
17 }
18
19 pub struct BasicBlock {
20     pub id: usize,
21     pub instr_range: Range<usize>,
22     pub terminator: Terminator,
23 }
24
25 pub enum Terminator {
26     Branch {
27         branch_instr_idx: usize,
28         taken_block: usize,
29         fallthrough_block: usize,
30     },
31     Fallthrough(usize),
32     Exit,
33 }

```

Листинг 3.7 — Структуры данных модуля asm_parse.rs (Rust)

Функция `parse_asm` сканирует текст построчно: при нахождении метки `function_name`: начинает сбор инструкций; при встрече следующей внешней метки (без точки) завершает разбор; внутрифункциональные метки вида `.LBV0_2`: записываются в `labels` с текущим индексом инструкции и впоследствии используются функцией `build_cfg` для выявления границ блоков.

Функция `instruction_to_chc` сопоставляет каждую инструкцию с именем соответствующего сгенерированного правила и списком операндов. Порядок операндов определяется порядком полей в кортеже `Sail` для каждого класса инструкций (листинг 3.8).

```

1  match op {
2      // I-type: addi rd, rs1, imm в (imm_bv12, rs1_idx,
   rd_idx)
3      "addi" | "addiw" | "andi" | "ori" | "xori" | "slti"
   | "sltiu" => {
4          let rd = reg_to_smt(&instr.operands[0])?;
5          let rs1 = reg_to_smt(&instr.operands[1])?;
6          let imm = imm_to_bv12(imm_val);
7          Ok((op.to_string(), vec![imm, rs1, rd]))
8      }
9      // Store: sd rs2, off(base) в (off_bv12, rs2_idx,
   base_idx)
10     "sb" | "sh" | "sw" | "sd" => {
11         let rs2 = reg_to_smt(rs2_name)?;
12         let base = reg_to_smt(base_name)?;
13         Ok((op.to_string(), vec![imm_to_bv12(offset), rs2,
   base]))
14     }
15     // Load: ld rd, off(base) в (off_bv12, base_idx,
   rd_idx)
16     "lb" | "lbu" | "lh" | "lhu" | "lw" | "lwu" | "ld"
   => {
17         let rd = reg_to_smt(rd_name)?;
18         let base = reg_to_smt(base_name)?;
19         Ok((op.to_string(), vec![imm_to_bv12(offset), base,
   rd]))
20     }

```

```

21      // R-type: add rd, rs1, rs2 в (rs2_idx, rs1_idx,
      rd_idx)
22      "add" | "sub" | "and" | "or" | "xor" | "slt" |
      "sltu" |
23      "sll" | "srl" | "sra" | "addw" | "subw" | "mul"
      | ... => {
24          let rd = reg_to_smt(&instr.operands[0])?;
25          let rs1 = reg_to_smt(&instr.operands[1])?;
26          let rs2 = reg_to_smt(&instr.operands[2])?;
27          Ok((op.to_string(), vec![rs2, rs1, rd]))
28      }
29      // Псевдоинструкция ret
30      "ret" => Ok(("jalr".to_string(),
31                  vec!["_ bv0 12".into(),
32                      "reg_ra".into(),
33                      "reg_zero".into()]]),

```

Листинг 3.8 — Отображение ассемблерных инструкций в SMT-операнды

Соответственно, формирование запроса — это связывание всех инструкций функции через промежуточные переменные состояния:

$$(regs_0, mem_0, pc_0) \xrightarrow{instr_1} (regs_1, mem_1, pc_1) \xrightarrow{instr_2} \dots \xrightarrow{instr_n} (regs_n, mem_n, pc_n)$$

где n — количество инструкций функции, а каждая тройка $(regs_i, mem_i, pc_i)$ — это промежуточное состояние машины после i -й инструкции.

Итоговое СНС-правило для блока функции формируется с инлайнингом семантики инструкций — без промежуточных СНС-отношений. Листинг 3.9 показывает фактически сгенерированное RICOVER правило для блока `func_361_bb0` из бенчмарка Csmith (функция `src`, 10 инструкций, 1 базовый блок).

```

1  (declare-rel func_361_bb0
2    ((Array (_ BitVec 5) (_ BitVec 64))
3     (Array (_ BitVec 64) (_ BitVec 8)) (_ BitVec 64)
4     (Array (_ BitVec 5) (_ BitVec 64))
5     (Array (_ BitVec 64) (_ BitVec 8)) (_ BitVec 64)))
6
7  (rule

```

```

8      (forall
9        ;; 11 переменных regs0..regs10, только 5 переменных
        mem0..mem4
10      ((regs0 ...) (mem0 ...) (pc0 ...))
11      (regs1 ...)          (pc1 ...)
12      (regs2 ...) (mem1 ...) (pc2 ...)
13      ...
14      (regs10 ...)          (pc10 ...))
15    (=> (and
16      ; addi sp, sp, -32
17      (= regs1 (set_reg regs0 reg_sp (bvadd (get_reg
        regs0 reg_sp)
18          ((_ sign_extend 52) (_
        bv4064 12)))))
19      (= pc1 (bvadd pc0 (_ bv4 64)))
20      ; sd ra, 24(sp)
21      (= regs2 regs1)
22      (= mem1 (write_mem_dword mem0
23          (bvadd (get_reg regs1 reg_sp)
24              ((_ sign_extend 52) (_
        bv24 12))))
25          ((_ extract 63 0) (get_reg regs1
        reg_ra))))
26      (= pc2 (bvadd pc1 (_ bv4 64)))
27      ; ... остальные инструкции ...
28      ; ret = jalr x0, 0(ra)
29      (= regs10 (set_reg regs9 reg_zero (bvadd pc9 (_
        bv4 64))))
30      (= pc10 (concat ((_ extract 63 1)
31          (bvadd (get_reg regs9
        reg_ra)
32              ((_
        sign_extend
        52) (_ bv0
        12)))))
33          (_ bv0 1))))
34      (func_361_bb0 regs0 mem0 pc0 regs10 mem4 pc10))))

```

Листинг 3.9 — Сгенерированное CHC-правило func_361_bb0 (SMT-LIB2, вывод RICOVER)

В блоке `forall` видна оптимизация устранения переменных памяти: из 10 инструкций лишь 4 (`sd`, `sd`, `sw`, `sh`) записывают память — вводятся переменные `mem1–mem4`. Инструкции `addi`, `ld` и `ret` не получают собственной переменной `memN`. Константа `(_ bv4064 12)` — 12-битное дополнение до двух для -32 (поскольку $4096 - 32 = 4064$).

Оптимизированная версия `func_362` (8 инструкций, `sw/sh` удалены `instcombine`) содержит только `mem0–mem2`: оба мёртвых сохранения исчезли вместе со своими переменными памяти. СНС-решатель Z3 отвечает `unsat` за 27 секунд: мёртвые записи выполнялись в приватный фрейм $[sp_0 - 32, sp_0)$, который исключён из `ObsMem`, и ABI-видимые регистры (`a0`, `ra`, `sp`, `s0`) совпадают в обоих вариантах.

3.4 Стандартная библиотека СНС

Стандартная библиотека определяет примитивы, общие для всех сгенерированных запросов. Операции над регистрами реализованы как SMT-функции (листинг 3.10).

```

1  (define-fun get_reg ((regs (Array (_ BitVec 5) (_ BitVec
2                                     (idx (_ BitVec 5))) (_
                                     BitVec 64)))
3      (ite (= idx (_ bv0 5))
4          (_ bv0 64)
5          (select regs idx)))
6
7  (define-fun set_reg ((regs (Array (_ BitVec 5) (_ BitVec
8                                     (idx (_ BitVec 5))
9                                     (val (_ BitVec 64))) (Array
                                     (_ BitVec 5) (_ BitVec 64)))
10      (ite (= idx (_ bv0 5))
11          regs
12          (store regs idx val)))

```

Листинг 3.10 — Операции `get_reg` и `set_reg` в `stdlib` (SMT-LIB2)

Функция `get_reg` возвращает 0 при чтении регистра с индексом 0 (`x0` — особый регистр в RISC-V), `set_reg` игнорирует запись при `idx = 0`. Это гарантирует семантику нулевого регистра без дополнительных ограничений в теле каждого СНС-правила.

Помимо регистровых операций, стандартная библиотека содержит:

- **Операции с памятью:** `mem_read_N` (сырое чтение N байт), `read_mem_word/read_mem_dword` (знаковое расширение до 64 бит), `write_mem_word/write_mem_dword` (запись в little-endian порядке), аналоги для 1- и 2-байтовых операций;
- **Константы ABI:** `reg_zero = 0b000000`, `reg_ra = 0b00001`, `reg_sp = 0b00010`, `reg_s0 = 0b01000`, `reg_a0 = 0b01010`.

3.5 Модель наблюдаемого состояния

Приведённая ранее формулировка проекционной эквивалентности $\approx_{\text{proj}}$ (формула 2.7) требует точного определения множества $\text{ObsMem}(\sigma_0)$ — наблюдаемой памяти.

При разборе функции `check-equiv` ищет первую инструкцию вида `addi sp, sp, -N` (или `c.addi sp, -N`). Число N является размером стекового фрейма функции. Приватный фрейм определяется как интервал $[\text{sp}_0 - N, \text{sp}_0)$, где sp_0 — значение указателя стека в начальном состоянии.

СНС-запрос `bad` эмитируется как набор правил, реализующих формулы из раздела 2.4. Для каждого ABI-видимого регистра (`a0`, `ra`, `sp`, `s0`) и счётчика команд (`pc`) генерируется отдельное правило, проверяющее расхождение; дополнительное правило проверяет наблюдаемую память. Листинг 3.11 показывает фрагмент сгенерированного RICOVER запроса для бенчмарка `bench1`.

```

1  (declare-rel bad ())
2
3  ; Расхождение ABI-регистра a0
4  (rule
5    (forall ((regs0 ...) (mem0 ...) (pc0 ...)
6              (regs1 ...) (mem1 ...) (pc1 ...)
7              (regs2 ...) (mem2 ...) (pc2 ...)
8              (sp0 (_ BitVec 64))))
9    (=> (and
10       (= sp0 (get_reg regs0 reg_sp))
11       (bvuge sp0 (_ bv0 64))
12       (bench1_BAD1 regs0 mem0 pc0 regs1 mem1 pc1)
13       (bench1_BAD2 regs0 mem0 pc0 regs2 mem2 pc2)
14       (not (= (get_reg regs1 reg_a0)
15              (get_reg regs2 reg_a0))))
16       bad)))

```

```

17
18 ; ... аналогичные правила для ra, sp, s0, pc ...
19
20 ; Расхождение наблюдаемой памяти
21 (rule
22   (forall ((regs0 ...) (mem0 ...) (pc0 ...)
23            (regs1 ...) (mem1 ...) (pc1 ...)
24            (regs2 ...) (mem2 ...) (pc2 ...)
25            (sp0 (_ BitVec 64)) (a (_ BitVec 64))))
26   (=> (and
27       (= sp0 (get_reg regs0 reg_sp))
28       (bvuge sp0 (_ bv0 64))
29       (bench1_BAD1 regs0 mem0 pc0 regs1 mem1 pc1)
30       (bench1_BAD2 regs0 mem0 pc0 regs2 mem2 pc2)
31       (obs_addr sp0 a)
32       (not (= (select mem1 a) (select mem2 a))))
33   bad)))
34
35 (query bad)

```

Листинг 3.11 — Сгенерированный запрос bad для бенчмарка bench1 (SMT-LIB2, вывод RICOVER)

Предикат `obs_addr` выводим тогда и только тогда, когда адрес `a` находится вне приватного фрейма `[sp0 - N, sp0)`, то есть является наблюдаемым. Если `bad` выводимо — найден контрпример; если нет — обе программы семантически эквивалентны относительно $\underset{\text{proj}}{\simeq}$.

3.6 Кодирование ветвлений и граф потока управления

Функции с условными переходами требуют многоблочного СНС-кодирования. RICOVER строит граф потока управления (CFG) и кодирует его через два слоя СНС-отношений.

Построение CFG. Функция `build_cfg` в `asm_parse.rs` делит функцию на базовые блоки: границы блоков проставляются на начало функции, на инструкцию-цель каждого перехода и на инструкцию сразу после каждой ветки. Блоки отделены друг от друга «терминаторами»: `Branch` (условный переход с адресом обоих путей), `Fallthrough` (непосредственный переход к следующему блоку) и `Exit` (блок заканчивается инструкцией `ret`).

Для каждого базового блока `bb<N>` функции `f` генерируются:

- 1) **Тело блока** — `f_bb<N>(state_in, state_out)`: цепочка ограничений для инструкций блока (без инструкции-терминатора);
- 2) **Композиция** — `f_from_bb<N>(state_in, state_out)`: описывает все пути выполнения от входа в блок `bb<N>` до завершения функции. Для входного блока (`bb0`) эта функция именуется просто `f`.

Такая структура позволяет естественно моделировать обратные рёбра (циклы) через рекурсивные СНС-правила: цикл-заголовок получает рекурсивное правило для `f_from_bb<N>`, а СНС-решатель находит индуктивный инвариант через фиксированную точку.

Рассмотрим функцию `src` (листинг 3.12).

1	<code>src:</code>		asm
2	<code>lw</code>	<code>a3, 0(a2)</code>	; bb0: загрузка из памяти
3	<code>lw</code>	<code>a4, 0(a1)</code>	; bb0: загрузка из памяти
4	<code>blt</code>	<code>a3, a4, .LBB0_2</code>	; bb0: терминатор - условный переход
5	<code>mv</code>	<code>a1, a2</code>	; bb1: fallthrough
6	<code>.LBB0_2:</code>		
7	<code>ld</code>	<code>a1, 0(a1)</code>	; bb2: загрузка двойного слова
8	<code>sd</code>	<code>a1, 0(a0)</code>	; bb2: сохранение в память
9	<code>ret</code>		; bb2: выход

Листинг 3.12 — Функция `src` из бенчмарка PR44306 с аннотацией базовых блоков (RISC-V ассемблер)

CFG: `bb0` в (`blt` взята) в `bb2 [exit]`; `bb0` в (`blt` не взята) в `bb1` в `bb2 [exit]`.

Функция `build_cfg` разделяет функцию на три блока. `bb0` содержит две инструкции `lw` (тело) и ветку `blt` (терминатор); `bb1` — одну инструкцию `mv` (терминатор `Fallthrough(2)`); `bb2` — три инструкции `ld`, `sd`, `ret` (терминатор `Exit`). Листинг 3.13 показывает фактически сгенерированные правила CFG-композиции.

1	<code>;; Входной блок bb0, ветвь взята (blt a3, a4): тело bb0 -> from_bb2</code>
2	<code>(rule</code>
3	<code>(forall ((regs0 ...) (mem0 ...) (pc0 ...)</code>
4	<code>(regs1 ...) (mem1 ...) (pc1 ...)</code>
5	<code>(regs2 ...) (mem2 ...) (pc2 ...))</code>
6	<code>(=> (and (PR443061_bb0 regs0 mem0 pc0 regs1 mem1 pc1)</code>

```

7          (bvslt (get_reg regs1 (_ bv13 5))
8              (get_reg regs1 (_ bv14 5)))
9          (PR443061_from_bb2 regs1 mem1 pc1 regs2
10             mem2 pc2))
11      (PR443061 regs0 mem0 pc0 regs2 mem2 pc2))))
12 ;; Входной блок bb0, ветвь не взята: тело bb0 -> from_bb1
13 (rule
14     (forall ((regs0 ...) (mem0 ...) (pc0 ...)
15             (regs1 ...) (mem1 ...) (pc1 ...)
16             (regs2 ...) (mem2 ...) (pc2 ...))
17         (=> (and (PR443061_bb0 regs0 mem0 pc0 regs1 mem1 pc1)
18                 (not (bvslt (get_reg regs1 (_ bv13 5))
19                             (get_reg regs1 (_ bv14
20                                     5)))))
21             (PR443061_from_bb1 regs1 mem1 pc1 regs2
22                 mem2 pc2))
23         (PR443061 regs0 mem0 pc0 regs2 mem2 pc2))))

```

Листинг 3.13 — Правила CFG-композиции функции src из PR44306 (SMT-LIB2)

Условие ветвления (`bvslt ...`) читает регистры из состояния **после** выполнения тела блока (`regs1`), а не из входного состояния (`regs0`): сначала исполняются инструкции-данные блока, затем вычисляется условие перехода. Это соответствует семантике инструкции `blt`: условие проверяется на актуальных значениях регистров в момент ветвления.

Все `declare-rel` эмитируются до всех `rule` — это требование Z3 Fixedpoint API: при обработке правила каждый предикат, используемый в теле, уже должен быть объявлен. `emit_composition_rules` поэтому делает два прохода: сначала объявляет все отношения, затем эмитирует правила.

ГЛАВА 4. ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА

Данная глава содержит описание тестовой среды, результаты верификации на ручных бенчмарках из базы Alive2, анализ производительности СНС-решателей и результаты масштабной оценки с помощью генератора случайных программ Csmith.

4.1 Тестовая среда и инструментарий

Все эксперименты проводились на рабочей станции, конфигурация которой приведена в таблице 2.

Таблица 2 — Конфигурация тестового стенда

Параметр	Значение
Процессор	12th Gen Intel Core i5-1235U
Оперативная память	8 ГБ
Операционная система	Arch Linux (ядро Linux 6.18.7-arch1-1, x86_64)
СНС-решатель	Z3 4.15.4 (64-бит)
Реализация RICOVER	Rust 1.89.0-nightly (edition 2024)
Компилятор LLVM/Clang	23.0.0git (цель: riscv64-linux-gnu)
Генератор программ Csmith	версия 2.4.0

Стратегия СНС-решателя — Spacer, параметр `set-logic HORN`. Тайм-аут Z3 для экспериментов с Csmith — 300 секунд (5 минут) на одну задачу.

4.2 Тестовая база: ошибки мисоптимизации из Alive2

Для первичной верификации корректности RICOVER использовалась тестовая база, составленная из подтверждённых ошибок мисоптимизации компилятора LLVM, обнаруженных инструментом Alive2 [2]. Каждая такая ошибка сопровождается тремя элементами:

- 1) исходный вариант кода (до оптимизации);
- 2) *ошибочная* оптимизация (с мисоптимизацией);
- 3) *исправленная* оптимизация (корректный вариант).

Методика верификации: RICOVER должен вернуть `sat` (расхождение обнаружено) для пары «до / ошибочная оптимизация» и `unsat` (эквивалентность подтверждена) для пары «до / исправленная оптимизация». Только оба результата вместе подтверждают корректность работы инструмента.

Воспроизведение ошибок Alive2 на уровне RISC-V ассемблера требовало дополнительного шага: исходные ошибки описаны на уровне

LLVM IR, и их необходимо воспроизвести путём компиляции конкретным LLVM-бэкендом для RISC-V.

В директории `benchmark/supported/` представлены следующие верифицированные бенчмарки:

- `bench1_BAD.s` — ошибочная оптимизация (ожидаемый результат: `sat`);
- `constant_folding_simple_SUCCESS.s` — корректная константная свёртка (`unsat`);
- `compressed_equiv_SUCCESS.s` — эквивалентность со сжатыми инструкциями (`unsat`);
- `amo_swap_SUCCESS.s` — атомарная обменная операция (`unsat`);
- `PR44306_BAD.s` — воспроизведение бага PR44306 из трекера LLVM (`sat`).

Результаты приведены в таблице 3.

Таблица 3 — Результаты верификации ручных бенчмарков

Бенчмарк	Тип	Инструкций (src/tgt)	Результат Z3	Время решения
<code>constant_folding</code>	UNSAT	14 / 2	unsat	< 1 с
<code>compressed_equiv</code>	UNSAT	10 / 8	unsat	< 1 с
<code>amo_swap</code>	UNSAT	6 / 4	unsat	< 1 с
<code>bench1</code>	SAT (ошибка)	12 / 8	sat	< 2 с
<code>PR44306</code>	SAT (ошибка)	18 / 10	sat	< 2 с

Все поддерживаемые бенчмарки дают ожидаемый результат. Ручные бенчмарки разрешаются менее чем за 2 секунды: они имеют небольшое число инструкций и не затрагивают агрессивных операций с памятью, которые усложняют задачу для массивной теории.

В бенчмарке `constant_folding` функция `foo1` вычисляет $y = x + 2$ при $x = 1$ через стек (14 инструкций); функция `foo2` возвращает `addi a0, zero, 3; ret` (2 инструкции). Z3 отвечает `unsat` мгновенно.

В бенчмарке `bench1 (sat)` воспроизводит конкретный баг LLVM. Z3 отвечает `sat` и предоставляет конкретное начальное состояние (модель), в котором оптимизированный вариант вычисляет иной результат, что позволяет воспроизвести ошибку без перебора тестовых входных данных.

4.3 Производительность СНС-решателей

На генерируемых Csmith функциях производительность Z3 существенно зависит от сложности СНС-задачи. Ключевой метрикой сложности является не общее число инструкций, а **число операций с памятью** (load/store: `lb`, `lh`, `lw`, `ld`, `sb`, `sh`, `sw`, `sd`) в исходной функции. Каждая

такая операция порождает ограничение над теорией массивов ($\text{Array}(\text{BitVec } 64 \rightarrow \text{BitVec } 8)$), которое является основным источником вычислительной сложности для решателя.

Таблица 4 — Зависимость результата Z3 от числа операций с памятью (20 программ, instcombine, тайм-аут 300 с)

Операций с памятью (src)	Протестировано	Решено	Тайм-аут	Доля решённых
3–8	4	4	0	100%
9–11	7	7	0	100%
12	1	1	0	100%
13+	5	0	4	0%

Практическая граница разрешимости — **около 12 операций с памятью** в исходной функции. Чисто регистровые функции решаются за секунды независимо от числа инструкций.

Два фактора, расширяющие границу разрешимости:

- 1) **Оптимизация устранения переменных памяти.** До оптимизации функция из 20 инструкций порождала 43 ограничения над массивами; после оптимизации — 10. Снижение числа агау-ограничений в 4,3 разакратно ускоряет поиск инварианта.
- 2) **Инлайнинг правил инструкций.** Вместо обращения к инструкционным отношениям через `declare-rel` тела правил встраиваются непосредственно в программное правило, устраняя слой вложенности СНС-системы и упрощая работу алгоритма Spacer [7].
- 3) **Пересылка store–load (store-load forwarding).** Код уровня O0 порождает обильные паттерны вида «записать значение в стек — немедленно считать из того же слота»:

```
1 sw a0, -20(s0) ; записать a0 в слот стека
2 lw a0, -20(s0) ; считать обратно (избыточно)
```

Каждое чтение из памяти порождает ограничение `read_mem` над массивной теорией. RICOVER выполняет **пересылку store–load** на этапе эмиссии СНС: если инструкция загрузки (`lw`, `ld`, ...) читает из слота, в который непосредственно перед этим была выполнена запись (`sw`, `sd`, ...) с тем же смещением, базовым регистром и шириной, ограничение `read_mem` заменяется прямым выражением над регистром-источником. Пересылка консервативна: она отменяется при модификации базового или исходного регистра, а также при любой записи в память по другому базовому адресу (conservative aliasing).

Результаты оценки оптимизации пересылки store-load на 100 программах Csmith (проход instcombine, строгие флаги, тайм-аут 300 с) приведены в таблице 5.

Таблица 5 — Зависимость результата Z3 от числа операций с памятью (100 программ, instcombine, с пересылкой store-load)

Операций с памятью (src)	Протестировано	Решено	Тайм-аут	Доля решённых
3–5	8	8	0	100%
6–8	23	23	0	100%
9–11	22	21	1	95%
12–14	12	6	6	50%
15–17	16	4	12	25%
18–20	9	3	6	33%
21+	16	0	16	0%

Всего протестировано 126 функций: 60 EQUIV, 5 SAT (ложные срабатывания по предусловиям ABI), 42 тайм-аута, 19 пропущено (неподдерживаемые инструкции). Медиана времени решения: 63,9 с, максимум: 293,2 с.

Сравнение с базовой конфигурацией (без пересылки store-load) приведено в таблице 6.

Таблица 6 — Сравнение результатов с пересылкой store-load и без неё

Метрика	Без пересылки	С пересылкой	Улучшение
Практическая граница	≈ 12 мем. оп.	≈ 15–18 мем. оп.	+25–50%
Доля решённых (9–11 оп.)	100%	95%	≈ то же
Доля решённых (12–14 оп.)	≈ 0%	50%	новая возможность
Доля решённых (15–17 оп.)	0%	25%	новая возможность
Доля решённых (18–20 оп.)	0%	33%	новая возможность
Макс. решённых мем. оп.	12	20	+67%

Оптимизация расширяет границу разрешимости с ≈ 12 до ≈ 15 –18 операций с памятью, с отдельными функциями до 20 мем. оп. (например, func_48: 20 оп., решено за 293 с; func_21: 18 оп., решено за 275 с).

При включении указателей (--relaxed) генерируются функции с интенсивным использованием памяти. Результаты на 20 программах приведены в таблице 7.

Таблица 7 — Результаты с пересылкой store-load на расслабленных Csmith-программах (20 программ, instcombine, тайм-аут 300 с)

Операций с памятью (src)	Протестировано	Решено	Тайм-аут	Доля решённых
6–8	2	2	0	100%
9–11	5	4	1	80%
12–14	1	1	0	100%
18–20	3	0	3	0%
21–35	6	0	6	0%
48–62	4	2	2	50%
96+	3	0	1	0%

Всего 28 функций протестировано: 8 EQUIV, 1 SAT (ложное срабатывание), 13 тайм-аутов, 4 пропущено, 2 ошибки. Медиана: 122,6 с, p90: 256,4 с. Пересылка store-load значительно снижает число эффективных операций с памятью в целевой функции (большинство функций показывают 4 мем. оп. после пересылки), однако сложность исходной стороны остаётся доминирующим фактором. Практическая граница в расслабленном режиме — ≈ 12 операций с памятью в источнике.

4.3.1 Конфигурация решателя Spacer

Z3 при указании `set-logic HORN` автоматически активирует движок Spacer. В экспериментах использовалась версия Z3 4.15.4 с параметрами по умолчанию. Ключевые параметры Spacer приведены в таблице 8.

Таблица 8 — Параметры Z3 Spacer, используемые по умолчанию

Параметр	Значение	Описание
<code>spacer.global</code>	false	Стратегия глобального руководства (global guidance)
<code>spacer.mbqi</code>	true	Model-based quantifier instantiation
<code>spacer.q3</code>	true	Квантификаторная обработка
<code>spacer.use_array_eq_generalizer</code>	true	Обобщение равенств массивов
<code>xform.inline_eager</code>	true	Жадный инлайнинг отношений
<code>xform.inline_linear</code>	true	Инлайнинг линейных правил
<code>xform.inline_linear_branch</code>	false	Инлайнинг линейных ветвящихся правил
<code>xform.coalesce_rules</code>	false	Слияние правил
<code>xform.array_blast</code>	false	Развёртка массивов в скаляры

Для оценки влияния параметров Spacer на производительность было проведено сравнительное исследование 7 конфигураций на трёх наборах данных: ручные бенчмарки, строгие Csmith-функции и расслабленные Csmith-функции (с указателями). Результаты приведены в таблице 9.

Таблица 9 — Сравнение конфигураций Spacer (8 запросов, 8–17 операций с памятью, тайм-аут 300 с)

Конфигурация	Решено	Медиана	Примечание
baseline (по умолчанию)	5 / 8	117,9 с	—
spacer.global=true	5 / 8	118,0 с	≈ 0%
xform.inline_linear_branch=true	5 / 8	118,2 с	≈ 0%
xform.coalesce_rules=true	5 / 8	121,0 с	+3%
spacer.global + inline + coalesce	5 / 8	118,5 с	≈ 0%
array_blast=true	5 / 8	78,1 с *	+63% на простых
use_array_eq_generalizer=false	5 / 8	117,9 с	+12% на сложных

*Значение для набора с лёгкими запросами; на сложных запросах array_blast ухудшает производительность.

Ни одна конфигурация не изменила множество решаемых запросов: все 7 вариантов решили одни и те же 5 из 8 запросов. Вариация времени между конфигурациями (до $\pm 15\%$) меньше вариации между отдельными запросами (до $\pm 60\%$). Стратегия global guidance (spacer.global), на которую возлагались надежды, не оказала значимого влияния на структуру СНС, порождаемую RICOVER. Развёртка массивов (array_blast) активно вредит — преобразование Array(BitVec 64 → BitVec 8) в скалярные переменные экспоненциально увеличивает число переменных. Конфигурация по умолчанию является близкой к оптимальной для данного класса СНС-задач.

4.4 Масштабная верификация с помощью Csmith

Для систематической оценки применимости подхода к реальным компиляторным оптимизациям была разработана система автоматизированного тестирования на основе генератора случайных программ Csmith [1]. Конвейер csmith_ricover_e2e.py функционирует следующим образом (листинг 4.1).

```

1 csmith --no-safe-math --no-pointers --no-global-variables
2     -> program.c
3     -> clang -O0 -Xclang -disable-O0-optnone -emit-llvm -S
4     -> base.ll
5     -> llc -O1 -> base.s
6     -> opt -passes='<pipeline>' -S -> opt.ll -> llc -O1 ->
    opt.s
    -> пофункциональный diff (пропуск: call, relocations,
    >max-instrs)

```

```

7      -> RICOVER check-equiv -> .smt2
8      -> z3 -T:300 -> sat/unsat/timeout

```

Листинг 4.1 — Конвейер автоматизированного тестирования
csmith_ricover_e2e.py

Флаг `-Xclang -disable-00-optnone` необходим, чтобы `opt` проходы фактически срабатывали на `O0` IR. Флаг `--no-global-variables` позволяет избежать PC-relative relocations (`auipc/%pcrel`), которые RICOVER пока не поддерживает.

Тестируемые проходы LLVM: `instcombine`, `simplifycfg`, `gvn`, `dse`, `licm`, `indvars`, `loop-rotate`, `loop-unroll`, `loop-deletion` — всего 9 конвейеров.

Из 185 функций с различиями между базовым и оптимизированным вариантами (20 программ, проход `instcombine`):

Таблица 10 — Воронка отбора функций (проход `instcombine`, 20 программ)

Стадия фильтрации	Количество	Доля от различающихся
Функции с различиями	185	100%
Заблокированы инструкцией <code>call</code>	152	82%
Заблокированы PC-relative relocation	136	73%
Без вызовов и relocations	20	11%
В пределах размера + поддерживаемые инструкции	11	6%

За серию из 100 программ Csmith (проход `instcombine`, строгие флаги, тайм-аут 300 с, с пересылкой `store-load`) было протестировано 107 функций с различиями между базовым и оптимизированным вариантами. Репрезентативные примеры из различных диапазонов сложности приведены в таблице 11.

Таблица 11 — Репрезентативные результаты верификации функций Csmith (100 программ, `instcombine`, с пересылкой `store-load`)

Категория	Функция	src / tgt ин-струкций	Мем. оп. (src / tgt)	Результат Z3	Время
Быстрая (регистрационная)	func_5	41 / 6	19 / 2	unsat	0,1 с
Типичная	func_70	15 / 9	9 / 4	unsat	63,9 с
Граничная	func_21	35 / 9	18 / 4	unsat	274,5 с
Граничная	func_48	37 / 10	20 / 4	unsat	293,2 с
Ложное сраб. (ABI)	func_54	26 / 9	16 / 4	sat	0,2 с
Тайм-аут	func_11	20 / 10	12 / 4	timeout	300 с

Функция `func_5` (19 операций с памятью в источнике, 2 после пересылки) решается мгновенно — пересылка `store-load` устраняет почти все `aggau`-ограничения. Функция `func_48` (20 мем. оп.) — наиболее сложная из решённых, на границе тайм-аута (293 с). Результат `sat` для `func_54` — ложное срабатывание по предусловиям ABI (см. раздел «Ограничения»). Всего из 107 протестированных функций: 60 подтверждены эквивалентными (`unsat`), 5 ложных срабатываний (`sat`), 42 превысили тайм-аут (подробная разбивка по числу операций с памятью — в 5).

Результаты прогона по всем 9 проходам (4 программы):

Таблица 12 — Результаты прогона по всем 9 проходам LLVM (12 программ Csmith, тайм-аут 300 с)

Проход LLVM	Протестировано	EQUIV	Тайм-аут
<code>instcombine</code>	14	12	2
<code>simplifycfg</code>	12	10	2
<code>gvn</code>	12	10	2
<code>dse</code>	12	10	2
<code>licm</code>	12	10	2
<code>indvars</code>	12	10	2
<code>loop-rotate</code>	12	10	2
<code>loop-unroll</code>	12	10	2
<code>loop-deletion</code>	12	10	2
Итого	110	92	18

92 из 110 тестируемых случаев подтверждены как эквивалентные (84%). Ни одного `sat` (ошибки мисоптимизации) — все 9 проходов LLVM производят семантически корректные оптимизации. 18 тайм-аутов приходятся на две функции с 17 и 21 операцией с памятью, стабильно превышающие 300-секундный порог независимо от прохода.

При включении флага `--relaxed` (указатели разрешены, `--no-pointers` флаг Csmith снят) генерируются более сложные функции с более интенсивным использованием памяти. За серию из 30 программ (все 9 проходов, тайм-аут 300 с, `--max-mem-ops 21`) несколько функций оказались доступны для верификации, из-за отсеивания функций с релокациями и т.п.. Результаты приведены в таблице 13.

Таблица 13 — Результаты прогона по всем 9 проходам LLVM (`--relaxed`, 30 программ, тайм-аут 300 с)

Программа	Функция	src / tgt ин-струкций	Мем. оп. (src / tgt)	Проходов	Результат
prog_002	func_1	17 / 10	8 / 4	9	9/9 EQUIV
prog_004	func_77	15 / 11	8 / 4	9	9/9 EQUIV
prog_009	func_1	12 / 10	17 / 4	9	9/9 EQUIV
prog_004	func_93	30 / 16	21 / 8	9	9/9 тайм-аут
prog_006	func_58	28 / 10	24 / 8	9	9/9 тайм-аут

Функции с 6–8 операциями с памятью подтверждены эквивалентными по всем 9 проходам; функции с 22–24 операциями стабильно превышают тайм-аут на каждом из проходов. Результат согласуется с границей разрешимости, установленной в 5.

4.5 Ограничения и направления развития

Известные ограничения прототипа:

- 1) **Масштабируемость решателя.** Оптимизация пересылки store-load расширила границу сложности с ≈ 12 до ≈ 15 –18 операций с памятью в исходной функции (см. 6), однако функции выше этого предела по-прежнему стабильно превышают тайм-аут 300 секунд. Настройка параметров Spacer не меняет границу разрешимости (см. 9). Направление развития: применение Eldarica [8] как альтернативного СНС-решателя с иной стратегией поиска инварианта, а также дальнейшая редукция array-ограничений.
- 2) **Функции с вызовами (call).** Csmith генерирует функции, вызывающие другие функции программы. RICOVER верифицирует одиночные функции и не поддерживает межпроцедурный анализ. Направление: compositional verification, верификация пре-/постусловий вызываемых функций.
- 3) **PC-relative relocations.** Функции, использующие auipc/%pcrel для загрузки глобальных переменных, не поддерживаются. Направление: расширение модели состояния на сегмент глобальных данных.
- 4) **Ложные срабатывания (false positives).** Идентифицировано два класса:
 - *Предусловия ABI:* RICOVER не ограничивает начальные значения регистров вызывным соглашением. Функции с узкими типами параметров (uint8_t, int16_t) могут давать sat, потому что O0 явно расширяет знак, а оптимизированная версия предполагает гарантию ABI.
 - *Мёртвые записи в стек по значению:* при передаче структуры по значению DSE корректно удаляет мёртвую запись в локальную

копию структуры, но эта копия находится за пределами приватного фрейма (на стеке вызывающей функции), и RICOVER считает её наблюдаемой.

ЗАКЛЮЧЕНИЕ

В данной работе разработана методология и реализован прототип инструмента RICOVER для автоматической формальной верификации семантической эквивалентности RISC-V ассемблерного кода до и после применения оптимизаций компилятора.

Достигнуты следующие результаты:

- 1) Разработан и обоснован метод автоматической генерации СНС-правил для инструкций RISC-V непосредственно из официальной спецификации Sail [6] через промежуточное представление Isla IR [11]. Метод устраняет необходимость ручного описания семантики и обеспечивает соответствие правил официальному стандарту архитектуры. Транслированы 199 из 746 вариантов инструкций, покрывающих полный профиль RV64IM.
- 2) Определена и обоснована нотация проекционной семантической эквивалентности ($\simeq_{\text{proj}}$), разграничивающая наблюдаемое состояние (ABI-видимые регистры, внешняя память) и приватное (стековый фрейм функции). Нотация принята как стандарт для всех экспериментов и позволяет корректно верифицировать типичные оптимизации компилятора — в частности, константную свёртку, устранение мёртвого кода, упрощение управляющего потока — без ложных отказов из-за различий в стековом layout.
- 3) Реализован сквозной конвейер формальной верификации: от архитектурной спецификации и пары ассемблерных файлов — к SMT-LIB2 запросу в формате set-logic HORN, совместимому с решателями Z3/Spacer [7] и Eldarica [8].
- 4) Верифицирован инструмент на тестовой базе из подтверждённых ошибок мисоптимизации LLVM [2] (bench1, PR44306 и другие): инструмент корректно возвращает sat для ошибочных оптимизаций и unsat для исправленных версий.
- 5) Проведена масштабная экспериментальная оценка с применением генератора случайных программ Csmith [1] и 9 конвейеров оптимизаций LLVM. Показано, что 7 из 10 малых функций (10–13 инструкций) подтверждаются как эквивалентные в пределах 300-секундного тайм-аута. Оптимизация устранения переменных памяти сократила число ограничений над массивами с 43 до 10 для функции из 20 инструкций, что кратно ускорило решение.

Основным практическим вкладом является демонстрация того, что подход перевода ассемблерной программы в СНС-формулу, дополненной семантической информацией формальной спецификации и передача формулы в SMT-решатель применим к реальным RISC-V программам, произведённым промышленным компилятором, без ручного описания семантики инструкций.

Направления дальнейших исследований:

- расширение СНС-кодирования на межпроцедурную верификацию;
- интеграция ограничений предусловий ABI для устранения ложных срабатываний на функциях с узкими типами параметров;
- перенос подхода на архитектуру AArch64, для которой существует аналогичная Sail-спецификация [11];
- исследование применения Eldarica [8] и других нелинейных СНС-решателей для улучшения масштабируемости.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Yang X. и др. Finding and understanding bugs in C compilers // SIGPLAN Not. New York, NY, USA: Association for Computing Machinery, 2011. Т. 46, № 6. Сс. 283–294.
2. Lopes N.P. и др. Alive2: bounded translation validation for LLVM // Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. Virtual, Canada: Association for Computing Machinery, 2021. Сс. 65–79.
3. Leroy X. Formal verification of a realistic compiler // Commun. ACM. New York, NY, USA: Association for Computing Machinery, 2009. Т. 52, № 7. Сс. 107–115.
4. Berger R. и др. Translation Validation for LLVM’s AArch64 Backend. New York, NY, USA: Association for Computing Machinery, 2025. Т. 9, № OOPSLA2.
5. Sammler M. и др. Islaris: verification of machine code against authoritative ISA semantics // Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation. San Diego, CA, USA: Association for Computing Machinery, 2022. Сс. 825–840.
6. RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA. 2019.
7. Hoder K., Bjørner N. Generalized property directed reachability // Proceedings of the 15th International Conference on Theory and Applications of Satisfiability Testing. Trento, Italy: Springer-Verlag, 2012. Сс. 157–171.
8. Hojjat H., Rümmer P. The ELDARICA Horn Solver // 2018 Formal Methods in Computer Aided Design (FMCAD). 2018. № . Сс. 1–7.
9. Bjørner N. и др. Horn Clause Solvers for Program Verification // Fields of Logic and Computation II: Essays Dedicated to Yuri Gurevich on the Occasion of His 75th Birthday / под ред. Beklemishev L.D. и др. Cham: Springer International Publishing, 2015. Сс. 24–51.
10. A structural approach to operational semantics // The Journal of Logic and Algebraic Programming. 2004. Сс. 17–139.
11. Armstrong A. и др. ISA semantics for ARMv8-a, RISC-v, and CHERI-MIPS // Proc. ACM Program. Lang. New York, NY, USA: Association for Computing Machinery, 2019. Т. 3, № POPL.

12. De Angelis E. и др. Semantics-based generation of verification conditions by program specialization // Proceedings of the 17th International Symposium on Principles and Practice of Declarative Programming. Siena, Italy: Association for Computing Machinery, 2015. Сс. 91–102.
13. Zhao J. и др. Formal verification of SSA-based optimizations for LLVM // SIGPLAN Not. New York, NY, USA: Association for Computing Machinery, 2013. Т. 48, № 6. Сс. 175–186.
14. Reid A. и др. Towards Making Formal Methods Normal: Meeting Developers where They Are // Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, 2020.
15. Tan G., Appel A.W. A compositional logic for control flow // Proceedings of the 7th International Conference on Verification, Model Checking, and Abstract Interpretation. Charleston, SC: Springer-Verlag, 2006. Сс. 80–94.